COMP 557, Assignment 4: Ray Tracing

In this assignment, you will be tasked with writing a ray tracer to render scenes described in json files. You are free to modify **any** part of the starter code. However, it should still be compatible with all the provided json files.

Requirements

This assignment has many of the same requirements as the previous, but needs two additional modules:

```
python -m pip install json5
python -m pip install taichi
```

We will be using a `json5` module to read scene files because it supports comments. If you are using VS Code, note that it default to assume JSON formatting of .json files, but you can set it up to use JSON with Comments.

Taichi is a domain-specific language embedded in Python that permits high-performance parallel programs. Using @ti.func and @ti.kernel and other decorations in your code, you can identify what parts should be compiled and optimized for parallel computation on GPU (vulkan, cuda, metal) or CPU. This power does not come without some restrictions, for example, we will use taichi `ti.vec3` and `ti.mat4` math typte rather than `glm` or `numpy`, and certain structures of python code are not permitted (error messages will let you know, or likewise it can make sense to read through the first 3 chapters of the docs: Why Taichi, Getting Started, Kernels & Functions).

Json scenes

Each json file defines a scene with key-value pairs to specify the image resolution, camera, lights, and a list of `objects`, i.e., geometry . The `objects` list may contain `"plane"`, `"sphere"`, `"mesh"` for triangle mesh, `"box"` for axis aligned box, hierarchies defined by using the `"node"` type, and instances defined by using the `"instance"` type along with a `"ref"` to a named node (the parser only supports instances defined at the root level). See `BoxStacks.json` for an example hierarchy and instances, but note that instances are primarily for scene definition as they generate copies of the other objects.

Scene nodes and geometry each have an associated transformation and a list of `children` geometry objects. The node definition can contain transformation attributes: `"position"` for translation, `"rotation"` for XYZ Euler rotation, and `"scale"` for uniform and non-uniform scale (and of course others, if you choose to add them). These transformations are composed with one another, in this order, to build the transformation (see the `load_geometry_transformation_matrix` function in `parser.py`). Recall that instances are like regular grouping nodes, but their transformation will be composed with the transformations of all the children of the referenced node, and copies of these children will be added to the objects in the scene.

Look at the provided examples and the parser to get a better idea of how the scene description files are organized. You will ***very likely*** need to define and implement additional attributes for extra objectives. You

are ***strongly encouraged*** to make new test scenes or modify the provided scenes to debug your implementation.

Running the Code

The `main.py` defines the entry point and requires at least one command line parameter. For example,

```
python main.py --help
python main.py -i Sphere.json
```

By default, no window is opened, but running with `-s` option will show the render as the samples are accumulated over multiple render passes. Note that after the scene is loaded, you can expect significant delay before the render kernel starts. This is because Taichi compiles the kernels and functions used to implement the assignment, and the more code there is the longer this will take. Taichi does have a compile cache enabled by default, but will not see an advantage from the cache if you are making edits to your code (likewise delays can be expected when switching scenes when providing multiple input files).

Use the `-ti` option to change the target architecture. Using `cuda` `vulcan` and `metal` should provide an important speed-up in comparison to `cpu`, which is the default. Note when running `cpu`, the render kernel will print a dot for every 10 lines completed, but otherwise, nothing will print when running on GPU.

Once the rendering passes are complete, the file is then saved to the output folder with the same name as the json file, but with a `.png` extension. Note that when using '-s', the file will be saved only on closing the gui window. By default, the file is to the `out` directory in the current directory. You can use the `-o` command line parameter to change the output dir.

When testing or debugging, you *may* want to examine a smaller image by using the `-f` command parameter to do the render at a factor of the specified resolution, e.g., `-f 0.1`. Given the highly parallelized taichi code, you may not see much speed-up except for very complicated scenes.

Note that you can provide multiple input files on the command line. For instance, to render all scenes you might do the following at a linux command prompt:

```
python main.py -i *.json
```

The equivalent command in windows powershell is the following:

```
python main.py -i (Get-ChildItem scenes/*.json).FullName
```

Provided Code

The `main.py` file will load the specified input json file(s), and call the render method of `scene.py`. This happens in a loop, where each iteration of render accumulates a running average colour for each pixel in the

image. This will be useful for anti-aliasing (with `jitter` set to true) or for other features that require multiple samples per pixel.

The `helperclasses.py` file provides you with basic classes that define Rays, Intersections, Lights, and Materials. Notice that these classes have `@ti.dataclass` decorations, to indicate that their contents will be used in taichi functions and kernels, and thus they contain taichi types such as `tm.vec3`. You can make any changes you like for your implementation, but add useful comments so that the TA can follow your changes and recall that your implementation must still be compatible with the provided test scenes.

The `geometry.py` file defines all the shapes that make up scenes: Sphere, Plane, AABox, and Mesh. Again, notice that these classes have `@ti.dataclass` decorations as they are used in `@ti.func` ray object intersection functions defined in the same file.

The intersection functions return an `Intersection`, which is likewise a `@ti.dataclass`, which contains a boolean to indicate if there is a hit or not, along with important information for processing the intersection (e.g., position, normal, material). See likewise that these intersection functions only check for intersection within the specified interval from `t_min` to `t_max`, as seen in class.

The `load_scene` function provided in `parser.py` file reads the provided json file and returns the `Scene`. If you want to describe new scene elements, you must modify the parser and the associated scene initialization. Any changes you make need to remain compatible with the **unmodified** json files provided to you (i.e., make your code backwards compatible!).

Ray Tracing Competition: best in show / le lapin d'or

You are encouraged to create a scene and an image to submit to the ray tracing competition (optional). This is an opportunity to show off all the features (e.g., extra features) of your ray tracer in an aesthetically pleasing novel scene that you design! Recall that participation in this competition is strongly encouraged should you think that you might later want to ask for a reference letter from the prof.

To participate, your assignment submission should include a ID-name-competition.json scene file for which you also create the image `ID-name-competition.png`, where ID is your student ID and name is your name. Your image should have a resolution of 1280 pixels horizontally by 720 pixels vertically. Also include a file `ID-name-competition.txt` containing a title and a short description of the technical achievements and artistic motivations for your entry. A small jury will judge submissions based on technical merit, creativity, and aesthetics. Results will be announced on the last day of class. You'll need to submit your assignment on time to guarantee that your submission is considered.

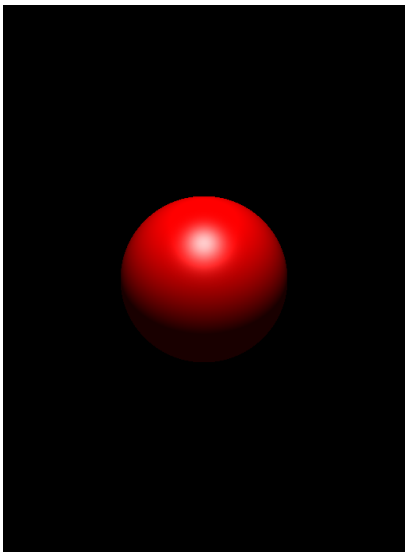
Objectives

1. Generate Rays (1 mark)

You are provided code that takes you up to the point where the scene is loaded and ready to render. You must finish code in `camera.py` to set up the camera and compute the rays for intersection with the scene. Use information provided in the `__init__` of the `Camera` class to build members you need to generate rays. Note that the constructor takes `glm` types, but ultimately you will store `tm` types to use in the `@ti-func create_ray`.

The camera ray through a given pixel should be *jittered* within the area of the pixel (uniformly) when the `jitter` argument is set to true. This permits many `samples` (a number specified in the json file) with rays going through different locations within the pixel to be averaged. This provides anti-aliaing of high frequency checkerboard textures that you will encounter later in the assignment.

2. Sphere Intersection (1 mark)



The `Sphere.json` scene includes a single sphere at the origin. Write the code to perform the sphere intersection and output black if there was no intersection, and white if there was one.

3. Lighting and Shading (3 marks)

In `scene.py` you will note code in function `intersect_scene` which will loop through all objects in the scene, keeping track of only the closest intersection (which contains information about the material and the normal). You might note a taichi `loop_config` command that tells taichi that there is no need to parallelize these loops over geometry because we are already letting it compute in parallel the different pixels.

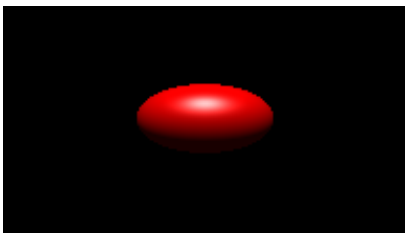
Implement the `compute_shading` function that uses the information returned from the `intersect_scene` to compute the colour of each pixel by summing the contribution of each of the lights defined in the json file. Currently only the ambient light is implemented.

You should implement diffuse Lambertian and Blinn-Phong specular illumination models as discussed in class. For specular surfaces, use `shininess` as the specular exponent. Lambertian surfaces can be defined by setting the `specular` colour to the zero vector (which is likewise the default value).

You need to handle both point lights and directional lights. See the `Light` class `ltype` member defined in `helperclasses.py` and the comments. Note that point lights should support the quadratic attenuation model seen in class. Also note, for convenient, the light definition in the json file has a `power` value, which simply modulates the colour and is an easy way to dim or brighten lights without changing the colour, or to simply boost the intensity of a light to account for quadratic attenuation (see `Plane2.json`, but you will need to complete the next objective first).

At right you can see the output of `Sphere.json` after properly implementing sphere intersections, lighting, and shading (though this is not an extensive test of directional and point lights with different attenuation models).

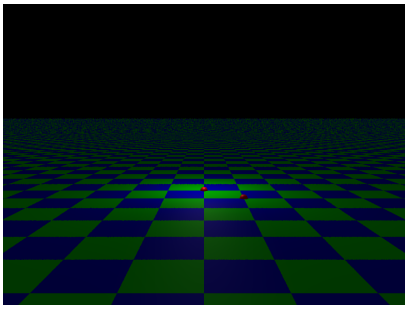
4. Geometry Transformations (1 mark)



Each scene object (e.g., the sphere, but likewise other objects, nodes, and instances) has a transform matrix to allow for modelling transformations. The transformations defined in each object, `M` and `M_inv` should transform the rays before intersecting the geometry. Use the inverse transformation to bring the mode into the local frame, then transform the intersection result to return to the caller. That is, `M` to transform points (though points can always be simply constructed by their t value on the ray), and the inverse transpose for normals.

You will need to add code for this in the sphere intersection function, but likewise in each of the other intersection functions you will write below. The image at right shows the result of scaling a sphere, as defined in the scene `Ellipse.json`.

5. Plane Intersection (1 mark)

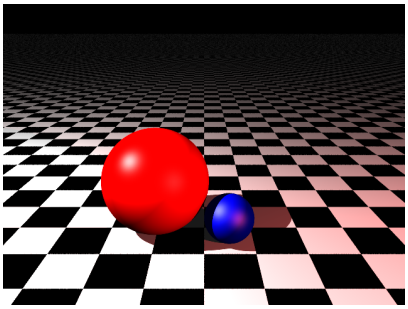


Implement the ray-plane intersection code. Planes are through the origin by default with the specified normal. Transformations are used to move planes away from the origin (or to scale and rotate them).

Planes are allowed to use two materials. If two materials are defined, then they create a checkerboard pattern, where each checker is a 1 x 1 square. Assuming that the plane contains the origin, we expect the point (0, 0, 0) to lie at the corner of 4 squares, where the squares in the +x +z and -x -z quadrants use the first material, while the squares in the +x -z and -x +z quadrants use the second material. You can test your code using `Plane.json` and `Plane2.json`.

At right, you can see the output that should be generated for `Plane2.json`.

6. Shadow Rays (1 mark)



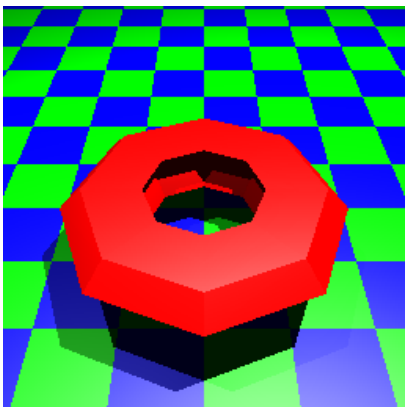
Modify your lighting code to compute a shadow ray in order to ensure that the light is not occluded before computing and adding this light's contribution. Make sure your shadows work with multiple lights. You can test this step using `TwoSpheresPlane.json`.

At right you can see the output that should be generated for `TwoSpheresPlane.json`.

7. Box Intersection (1 mark)

Add code in the AABB class that will calculate intersections for axis-aligned bounding boxes, which are defined in our code by the position of two diagonal corners (the min and max corners). You can test this step using `BoxRGBLights.json` and at the right margin you can see the output you should generate for this scene (the box has white reflectance properties, but is illuminated by a red, green, and blue light placed on x y and z axes).

8. Triangle Mesh Intersection (1 mark)



Implement code that will calculate intersections with a Mesh. Note that you do not have vertex normals by default, so flat shaded triangles are fine (though using interpolation of normals specified at vertices across triangles would be nice if you load meshes with such information, which is possible with `trimesh`).

Note that the parser packs all meshes into one list of vertices and faces, so each mesh references a subset of this data. This permits multiple meshes in a single scene, without trying to wrangle multiple datastructures in taichi.

You can test this step using `TorusMesh.json`, for which you can see sample output at right. Larger meshes, such as the bunny, benefit from some acceleration techniques (see 'choose your own adventure' objective below).

9. Choose your own adventure! (4 marks + max 5 marks bonus)

There is a total of 15 marks to obtain with this assignment. There are 10 in the previous objectives, and one in the last objective. In this objective you will choose features to implement to obtain the remaining 4 marks to have a full 15/15 on the assignment. You may likewise continue to collect another 5 bonus marks to have 20/15 on this assignment.

A combination of several additional features will be necessary to complete and then max out the bonus marks.

To receive marks, your features must be briefly described in the readme file, and clearly demonstrated to be correct with test scenes, result images. For anything beyond the list below, you should provide a brief justification for why your feature is worth so many marks. Ultimately the TAs and professor will evaluate the difficulty and assign additional marks accordingly.

Be sure to document all your additional features in your readme file!

- Sampling and Recursion
  - Mirror reflection and or Fresnel Reflection (0.5 marks)
  - Refraction (0.5 marks)
  - Motion blur (0.5 marks simple motion, 1 mark complex motion trajectories)
  - Depth of field blur (1 mark)
  - Area lights for soft shadows (1 mark)
  - Path tracing (requires emission materials, 2 marks)
- Geometry
  - Quadrics (cones, cylinders, etc.) (1 mark)
  - Ray marched implicits (metaballs, SDFs, etc.) (1.5 marks)
  - Bezier surface patches, e.g., a teapot! (2 marks)
  - Boolean operations for constructive solid geometry (probably hard in taichi, discuss on Ed) (2 marks?)
- Textures
  - Environment maps, i.e., use a cube map or sphere map (1 mark)
  - Textured mapped meshes (trimesh loads texture coords!) (1.5 marks)
  - Perlin or simplex noise for bump maps, or procedural volume textures (1 mark)
- Other
  - Acceleration techniques for BIG meshes (e.g., BIG bunny, dragon) (0.5 marks for single AABB, 1+ for advanced strategies and evaluation of benefits)
  - Something else totally awesome (discuss on boards, justify how many marks you want in the readme)

10. A Novel Scene / Competition Scene (1 mark)

Create a unique scene of your own. Be creative. Aim to have some amount of complexity to make it interesting (i.e., different shapes and different materials). Your scene should try to demonstrate all features of your ray tracer (see the 'choose your own adventure' objective). Be sure to include your name and student

number in the filename as described above in the competition information. That said, your novel scene need not be the same scene that you submit to the competition.

Finished?

Great! Submit your source code files and your renders of the test scenes as a zip file. Be sure the top of each file has your name and student number, and should there be any other information you wish to include put it in a readme file (e.g., request to use your late penalty waiver).

Note that you are encouraged to discuss assignments with your classmates, but not to the point of sharing code and answers. All code and written answers must be your own. Please see the course outline and the fine print in the slides of the first lecture.